

Local AI Software Team

Full Technical Design Document

1. Technical Summary

Local AI Software Team is a local-first developer tool that helps users turn rough software requirements into structured software delivery outputs.

The product has two main interfaces:

1. **CLI** — the main execution engine.
2. **Local Web UI** — settings, dashboard, run history, docs viewer, logs, reports, and approval screens.

The product does not require:

- desktop app
- backend cloud
- user login
- cloud database
- SaaS infrastructure

The system runs on the user's machine and stores project data locally.

It orchestrates existing AI coding tools such as:

- Claude Code
- Codex CLI
- Gemini CLI
- Aider
- Cursor Agent CLI
- OpenHands later
- other AI CLIs later

The system acts as a local AI software team:

- Business Analyst Agent
- Product Owner Agent
- Project Manager Agent
- Architect Agent
- Developer Agent
- QA Agent
- Code Reviewer Agent
- Security Reviewer Agent
- Reporter Agent

2. Core Technical Goal

The user should be able to run:

```
aiteam ui
```

Then open:

```
http://localhost:4317
```

From the local web UI, the user can:

- configure project settings
- configure AI CLI tools
- configure agent roles
- configure test commands
- configure safety rules
- input rough requirements
- run workflows
- review generated docs
- approve code generation
- view logs
- view test results
- view final reports

The user should also be able to run directly from CLI:

```
aiteam run "Thêm chức năng reset password bằng OTP email"
```

The system will produce:

```
rough requirement
→ clarified requirement
→ acceptance criteria
→ technical design
→ task breakdown
→ test matrix
→ implementation prompt
→ optional code patch
→ test result
→ review report
→ final delivery report
→ traceability matrix
```

3. Non-goals for MVP

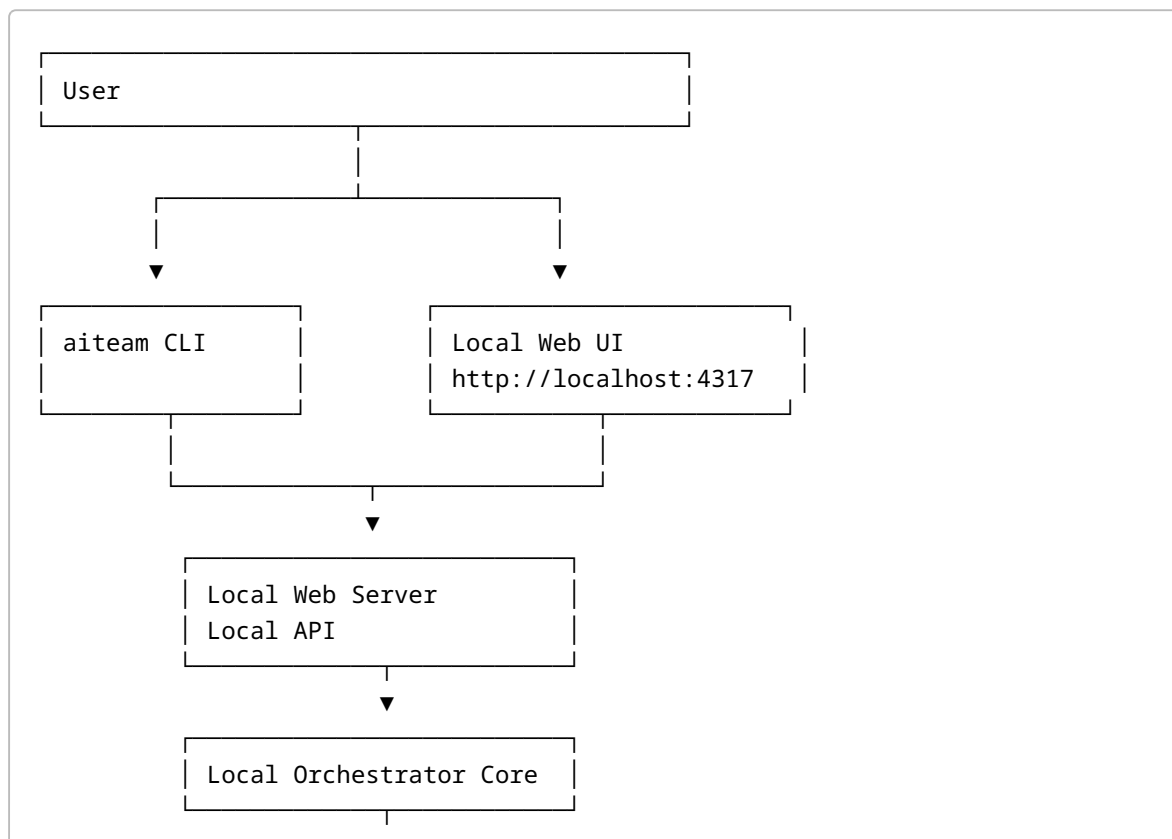
The MVP should not include:

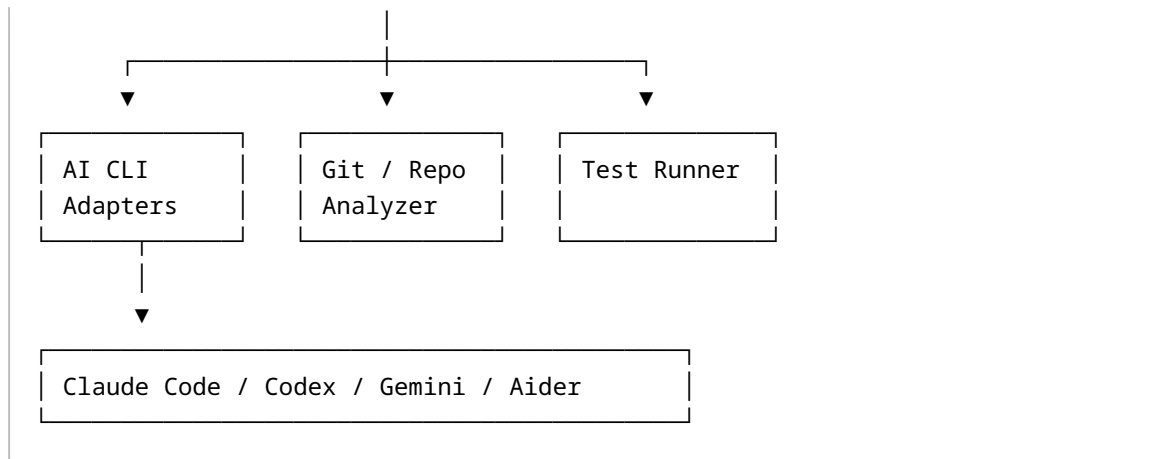
- desktop app
- cloud backend
- user login
- billing
- online team workspace
- automatic deployment
- full Slack event bot
- full Jira workflow automation
- enterprise SSO
- centralized admin console

The MVP should focus on:

- local CLI
- local web UI
- local project configuration
- local document generation
- local AI CLI orchestration
- local file-based artifacts
- local run history
- local safety controls

4. High-level Architecture





5. Recommended Tech Stack

5.1 Runtime

Use:

Node.js + TypeScript

Reason:

- easy to build CLI
- easy to run shell commands
- easy to start local web server
- easy to share types between CLI and local web UI
- easy to publish as npm package
- good ecosystem for Git, process execution, SQLite, and web UI

5.2 CLI Framework

Recommended:

Commander.js

Alternative:

oclif

Use Commander.js for MVP because it is simpler and faster.

5.3 Local Web UI

Recommended:

React + Vite + TypeScript

UI libraries:

Tailwind CSS
shadcn/ui optional
TanStack Query
React Router

5.4 Local Web Server

Recommended:

Fastify

Alternative:

Express

Fastify is preferred because it is lightweight, fast, and TypeScript-friendly.

5.5 Local Database

Recommended:

SQLite

Library:

better-sqlite3

Used for:

- projects
- runs
- artifacts metadata
- workflow status
- command logs
- agent tasks
- settings metadata
- integration metadata

5.6 Local Files

Use normal file system for generated artifacts:

- Markdown docs
- JSON outputs
- logs
- patches
- reports
- traceability files

SQLite stores metadata. Files store the actual generated content.

5.7 Process Runner

Recommended:

```
execa
```

Used for:

- running AI CLIs
- running Git commands
- running test commands
- streaming logs
- applying timeout
- capturing exit codes

5.8 Git Integration

Recommended:

```
simple-git
```

Used for:

- checking Git status
- collecting changed files
- creating diff patches
- creating branches later
- rollback support
- detecting current repository

5.9 Schema Validation

Recommended:

Zod

Used for:

- config validation
- workflow input validation
- AI output validation
- local API validation
- integration settings validation

6. Monorepo Structure

Recommended repository structure:

```
local-ai-software-team/  
  package.json  
  pnpm-workspace.yaml  
  tsconfig.base.json  
  
  apps/  
    cli/  
      src/  
        index.ts  
        commands/  
          init.command.ts  
          ui.command.ts  
          doctor.command.ts  
          run.command.ts  
          spec.command.ts  
          plan.command.ts  
          code.command.ts  
          test.command.ts  
          review.command.ts  
          report.command.ts  
  
    local-server/  
      src/  
        server.ts  
        routes/  
          settings.routes.ts  
          projects.routes.ts  
          runs.routes.ts  
          artifacts.routes.ts  
          logs.routes.ts  
          integrations.routes.ts  
  
    local-web/  
      src/  
        main.tsx
```

```
app/  
  App.tsx  
  router.tsx  
pages/  
  DashboardPage.tsx  
  SettingsPage.tsx  
  NewRequirementPage.tsx  
  RunsPage.tsx  
  RunDetailPage.tsx  
  PromptTemplatesPage.tsx  
  IntegrationsPage.tsx  
components/  
services/  
hooks/  
  
packages/  
  core/  
    src/  
      orchestrator/  
      workflows/  
      agents/  
      prompts/  
      policies/  
      traceability/  
      reports/  
      types/  
  
  adapters/  
    src/  
      ai/  
        ai-cli-adapter.ts  
        claude-code.adapter.ts  
        codex.adapter.ts  
        gemini.adapter.ts  
        aider.adapter.ts  
      git/  
        git.service.ts  
      shell/  
        shell-runner.ts  
      test/  
        test-runner.ts  
      integrations/  
        github.adapter.ts  
        jira.adapter.ts  
        slack.adapter.ts  
  
  storage/  
    src/  
      sqlite/  
        db.ts  
        migrations/
```



```
repositories/  
  project.repository.ts  
  run.repository.ts  
  artifact.repository.ts  
  setting.repository.ts  
  agent-task.repository.ts  
  
shared/  
  src/  
    types/  
    schemas/  
    constants/  
    utils/  
  
templates/  
  prompts/  
    ba-agent.md  
    po-agent.md  
    pm-agent.md  
    architect-agent.md  
    developer-agent.md  
    qa-agent.md  
    reviewer-agent.md  
    security-reviewer-agent.md  
    reporter-agent.md  
  
  project-types/  
    java-spring-boot.yaml  
    node-nestjs.yaml  
    react-vite.yaml  
  
docs/  
  idea.md  
  technical-design.md  
  roadmap.md
```

7. Local Project Data Layout

Inside each user project:

```
.ai-team/  
  config.json  
  database.sqlite  
  
  prompts/  
    ba-agent.md  
    po-agent.md  
    pm-agent.md  
    architect-agent.md
```

- developer-agent.md
- qa-agent.md
- reviewer-agent.md
- security-reviewer-agent.md
- reporter-agent.md

runs/

- 2026-06-22-143000-reset-password-otp/
 - input.md

requirement/

- clarified-requirement.md
- acceptance-criteria.md
- open-questions.md
- assumptions.md
- business-rules.md

design/

- technical-design.md
- api-design.md
- db-design.md
- service-flow.md
- risks.md

tasks/

- task-breakdown.md
- task-breakdown.json
- jira-ready-tasks.md

tests/

- test-matrix.md
- test-matrix.json
- test-result.md

implementation/

- implementation-plan.md
- implementation-prompt.md
- agent-output.log
- changed-files.json
- diff.patch

review/

- review-report.md
- security-review.md
- requirement-coverage.md

report/

- final-report.md
- traceability.md
- traceability.json

```
logs/  
  workflow.log  
  shell.log  
  agent.log
```

8. CLI Commands

8.1 init

Initialize `.ai-team` in the current repository.

```
aiteam init
```

Actions:

1. Detect Git repository.
2. Create `.ai-team` folder.
3. Create `config.json`.
4. Create SQLite database.
5. Copy default prompt templates.
6. Detect project type if possible.
7. Suggest build/test commands.

Options:

```
aiteam init --type java-spring-boot  
aiteam init --force
```

8.2 ui

Start local web server and local web UI.

```
aiteam ui
```

Default URL:

```
http://localhost:4317
```

Options:

```
aiteam ui --port 4317  
aiteam ui --open
```

8.3 doctor

Check local environment.

```
aiteam doctor
```

Checks:

- Node.js installed
- Git installed
- current folder is Git repo
- `.ai-team` initialized
- Claude Code installed
- Codex CLI installed
- Gemini CLI installed
- Aider installed
- Maven/Gradle/npm installed depending on project
- config valid
- SQLite accessible
- integration tokens available if enabled

Example output:

```
AI Team Doctor
```

```
Project: OK
Git: OK
Node: OK
Claude Code: OK
Codex CLI: Not found
Gemini CLI: OK
Aider: Not found
Maven: OK
Config: OK
Jira Token: Missing
Slack Token: Missing
```

8.4 run

Run workflow from raw requirement.

```
aiteam run "Thêm chức năng reset password bằng OTP email"
```

Options:

```
aiteam run "..." --mode docs-only
aiteam run "..." --mode assisted
aiteam run "..." --mode semi-auto --agent codex
aiteam run "..." --mode multi-agent
aiteam run "..." --max-iterations 3
aiteam run "..." --no-code
aiteam run "..." --no-test
```

8.5 spec

Generate clarified requirement and acceptance criteria.

```
aiteam spec --run <runId>
```

Output:

- clarified requirement
- business rules
- acceptance criteria
- open questions
- assumptions

8.6 plan

Generate design, task breakdown, and test matrix.

```
aiteam plan --run <runId>
```

Output:

- technical design
- API design
- DB design
- task breakdown
- test matrix
- implementation plan

8.7 code

Run selected AI coding CLI.

```
aiteam code --run <runId> --agent codex
```

Supported agent values:

```
claude  
codex  
gemini  
aider
```

Output:

- agent logs
- changed files
- diff patch
- implementation notes

8.8 test

Run configured test commands.

```
aiteam test --run <runId>
```

Options:

```
aiteam test --unit  
aiteam test --integration  
aiteam test --build  
aiteam test --lint
```

8.9 review

Review code changes and test result.

```
aiteam review --run <runId>
```

Output:

- review report
- security review
- requirement coverage report

8.10 report

Generate final delivery report.

```
aiteam report --run <runId>
```

Output:

- final report
- traceability matrix
- next steps
- remaining risks

9. Local Web UI

9.1 Local Web UI Purpose

The local web UI is used for:

- project settings
- AI CLI settings
- integration token setup
- prompt template editing
- workflow execution
- run history
- docs viewing
- logs viewing
- diff viewing
- test result viewing
- report viewing
- approval gates

9.2 Screens

Dashboard

Shows:

- current project
- latest runs
- latest workflow status
- enabled AI CLIs
- enabled integrations
- test command status
- safety mode

Settings

Sections:

- project settings
- AI CLI settings
- agent role mapping
- test/build commands
- safety settings
- documentation settings

- integration settings

New Requirement

Fields:

- raw requirement
- mode
- target project
- selected agent
- max iterations
- generate docs
- generate tasks
- run code
- run tests
- post result to external tools if enabled

Runs

Shows run history:

- title
- date
- mode
- status
- selected agent
- test status
- report status

Run Detail

Tabs:

- Input
- Requirement
- Design
- Tasks
- Tests
- Implementation
- Review
- Report
- Traceability
- Logs

Prompt Templates

Allows user to edit prompts for:

- BA Agent
- PO Agent

- PM Agent
- Architect Agent
- Developer Agent
- QA Agent
- Reviewer Agent
- Security Reviewer Agent
- Reporter Agent

Integrations

Configure:

- GitHub
- Jira
- Slack
- GitLab later
- Confluence later
- Notion later

10. Local API Routes

The local web UI talks to the local server.

Base URL:

```
http://localhost:4317/api
```

10.1 Settings Routes

```
GET    /api/settings
PUT    /api/settings
GET    /api/settings/effective
```

10.2 Project Routes

```
GET    /api/projects
POST   /api/projects
GET    /api/projects/:id
PUT    /api/projects/:id
DELETE /api/projects/:id
```

10.3 Run Routes

```
GET    /api/runs
POST   /api/runs
```

```
GET    /api/runs/:id
POST   /api/runs/:id/spec
POST   /api/runs/:id/plan
POST   /api/runs/:id/code
POST   /api/runs/:id/test
POST   /api/runs/:id/review
POST   /api/runs/:id/report
POST   /api/runs/:id/cancel
```

10.4 Artifact Routes

```
GET    /api/runs/:id/artifacts
GET    /api/runs/:id/artifacts/:artifactId
PUT    /api/runs/:id/artifacts/:artifactId
```

10.5 Logs Routes

```
GET    /api/runs/:id/logs
GET    /api/runs/:id/logs/stream
```

10.6 Diff Routes

```
GET    /api/runs/:id/diff
GET    /api/runs/:id/changed-files
```

10.7 Integration Routes

```
GET    /api/integrations
POST   /api/integrations/jira/test
POST   /api/integrations/slack/test
POST   /api/integrations/github/test
```

11. Configuration

Config file:

```
.ai-team/config.json
```

Example:

```
{
  "version": "1.0.0",
```

```

"project": {
  "name": "hotel-booking-service",
  "type": "brownfield",
  "language": "java",
  "framework": "spring-boot",
  "buildTool": "maven",
  "workingDir": "."
},
"agents": {
  "defaultBa": "gemini",
  "defaultPo": "gemini",
  "defaultPm": "gemini",
  "defaultArchitect": "claude",
  "defaultDeveloper": "codex",
  "defaultQa": "codex",
  "defaultReviewer": "claude",
  "defaultSecurityReviewer": "claude",
  "defaultReporter": "gemini"
},
"cli": {
  "claude": {
    "enabled": true,
    "command": "claude"
  },
  "codex": {
    "enabled": true,
    "command": "codex"
  },
  "gemini": {
    "enabled": true,
    "command": "gemini"
  },
  "aider": {
    "enabled": false,
    "command": "aider"
  }
},
"commands": {
  "build": "mvn clean package -DskipTests",
  "unitTest": "mvn test",
  "integrationTest": "mvn verify",
  "lint": ""
},
"safety": {
  "requireApprovalBeforeCode": true,
  "requireApprovalBeforeCommit": true,
  "requireApprovalBeforeExternalUpdate": true,
  "maxIterations": 3,
  "commandTimeoutSeconds": 900,
  "denyFiles": [
    ".env",

```

```

        ".env.*",
        "*.pem",
        "*.key",
        "credentials.json",
        "application-prod.yml",
        "application-production.yml"
    ],
    "warnFiles": [
        "pom.xml",
        "build.gradle",
        "package.json",
        "Dockerfile",
        ".github/workflows/*"
    ],
    "denyCommands": [
        "rm -rf /",
        "sudo",
        "chmod 777",
        "curl | sh",
        "wget | sh",
        "mkfs",
        "dd if="
    ]
},
"docs": {
    "generatePrd": true,
    "generateSrs": true,
    "generateTechnicalDesign": true,
    "generateApiDesign": true,
    "generateDbDesign": true,
    "generateTestMatrix": true,
    "generateTraceability": true
},
"integrations": {
    "github": {
        "enabled": false,
        "mode": "gh-cli",
        "owner": "",
        "repo": ""
    },
    "jira": {
        "enabled": false,
        "siteUrl": "",
        "email": "",
        "projectKey": "",
        "tokenRef": ""
    },
    "slack": {
        "enabled": false,
        "channelId": "",
        "tokenRef": ""
    }
}

```

```
}  
}  
}
```

12. Credential Handling

12.1 Credential Rule

Never store raw tokens directly in:

- `config.json`
- generated docs
- logs
- prompts
- reports
- Git-tracked files

12.2 Recommended Storage

Use OS keychain:

```
macOS: Keychain  
Windows: Credential Manager  
Linux: Secret Service
```

In Node.js, use a keychain library such as:

```
keytar
```

12.3 Token Reference Pattern

Config stores only token references:

```
{  
  "jira": {  
    "enabled": true,  
    "siteUrl": "https://company.atlassian.net",  
    "email": "user@example.com",  
    "projectKey": "ABC",  
    "tokenRef": "aiteam:jira:default"  
  }  
}
```

Actual token is stored in OS keychain.

12.4 Environment Variable Option

Advanced users may use environment variables:

```
export AITEAM_JIRA_TOKEN="..."
export AITEAM_SLACK_BOT_TOKEN="..."
export AITEAM_GITHUB_TOKEN="..."
```

Resolution priority:

1. Environment variable
2. OS keychain
3. Prompt user in local web UI

13. Core Domain Model

13.1 Project

```
type Project = {
  id: string;
  name: string;
  rootPath: string;
  type: "greenfield" | "brownfield";
  language?: string;
  framework?: string;
  buildTool?: string;
  createdAt: string;
  updatedAt: string;
};
```

13.2 Run

```
type Run = {
  id: string;
  projectId: string;
  title: string;
  rawRequirement: string;
  mode: RunMode;
  status: RunStatus;
  selectedAgent?: string;
  createdAt: string;
  updatedAt: string;
};
```

```
type RunMode =  
  | "docs-only"  
  | "assisted"  
  | "semi-auto"  
  | "multi-agent";
```

```
type RunStatus =  
  | "CREATED"  
  | "SPEC_GENERATING"  
  | "SPEC_GENERATED"  
  | "PLAN_GENERATING"  
  | "PLAN_GENERATED"  
  | "WAITING_FOR_APPROVAL"  
  | "CODING"  
  | "CODE_GENERATED"  
  | "TESTING"  
  | "TEST_PASSED"  
  | "TEST_FAILED"  
  | "REVIEWING"  
  | "REVIEW_PASSED"  
  | "REVIEW_FAILED"  
  | "REPORT_GENERATED"  
  | "FAILED"  
  | "CANCELLED";
```

13.3 Artifact

```
type Artifact = {  
  id: string;  
  runId: string;  
  type: ArtifactType;  
  name: string;  
  path: string;  
  format: "markdown" | "json" | "patch" | "log" | "text";  
  createdAt: string;  
  updatedAt: string;  
};
```

```
type ArtifactType =  
  | "RAW_REQUIREMENT"  
  | "CLARIFIED_REQUIREMENT"  
  | "ACCEPTANCE_CRITERIA"  
  | "OPEN_QUESTIONS"  
  | "ASSUMPTIONS"  
  | "TECHNICAL_DESIGN"  
  | "API_DESIGN"  
  | "DB_DESIGN"
```

```

| "TASK_BREAKDOWN"
| "TEST_MATRIX"
| "IMPLEMENTATION_PLAN"
| "IMPLEMENTATION_PROMPT"
| "AGENT_LOG"
| "DIFF_PATCH"
| "TEST_RESULT"
| "REVIEW_REPORT"
| "SECURITY_REVIEW"
| "FINAL_REPORT"
| "TRACEABILITY";

```

13.4 AgentTask

```

type AgentTask = {
  id: string;
  runId: string;
  role: AgentRole;
  adapter: AiAdapterName;
  promptPath: string;
  status: "PENDING" | "RUNNING" | "SUCCESS" | "FAILED" | "CANCELLED";
  startedAt?: string;
  finishedAt?: string;
  outputLogPath?: string;
};

```

13.5 AgentRole

```

type AgentRole =
| "BA"
| "PRODUCT_OWNER"
| "PROJECT_MANAGER"
| "ARCHITECT"
| "DEVELOPER"
| "QA"
| "CODE_REVIEWER"
| "SECURITY_REVIEWER"
| "DEVOPS"
| "REPORTER";

```

13.6 TraceabilityItem

```

type TraceabilityItem = {
  id: string;
  runId: string;
  requirementId: string;
};

```



```

    requirementText: string;
    acceptanceCriteriaIds: string[];
    taskIds: string[];
    codeFiles: string[];
    testCases: string[];
    testResults: string[];
    status: "COVERED" | "PARTIAL" | "NOT_COVERED";
};

```

14. SQLite Schema

14.1 projects

```

CREATE TABLE projects (
  id TEXT PRIMARY KEY,
  name TEXT NOT NULL,
  root_path TEXT NOT NULL,
  type TEXT NOT NULL,
  language TEXT,
  framework TEXT,
  build_tool TEXT,
  created_at TEXT NOT NULL,
  updated_at TEXT NOT NULL
);

```

14.2 runs

```

CREATE TABLE runs (
  id TEXT PRIMARY KEY,
  project_id TEXT NOT NULL,
  title TEXT NOT NULL,
  raw_requirement TEXT NOT NULL,
  mode TEXT NOT NULL,
  status TEXT NOT NULL,
  selected_agent TEXT,
  created_at TEXT NOT NULL,
  updated_at TEXT NOT NULL,
  FOREIGN KEY (project_id) REFERENCES projects(id)
);

```

14.3 artifacts

```

CREATE TABLE artifacts (
  id TEXT PRIMARY KEY,
  run_id TEXT NOT NULL,
  type TEXT NOT NULL,

```

```

name TEXT NOT NULL,
path TEXT NOT NULL,
format TEXT NOT NULL,
created_at TEXT NOT NULL,
updated_at TEXT NOT NULL,
FOREIGN KEY (run_id) REFERENCES runs(id)
);

```

14.4 agent_tasks

```

CREATE TABLE agent_tasks (
  id TEXT PRIMARY KEY,
  run_id TEXT NOT NULL,
  role TEXT NOT NULL,
  adapter TEXT NOT NULL,
  prompt_path TEXT NOT NULL,
  status TEXT NOT NULL,
  started_at TEXT,
  finished_at TEXT,
  output_log_path TEXT,
  FOREIGN KEY (run_id) REFERENCES runs(id)
);

```

14.5 command_runs

```

CREATE TABLE command_runs (
  id TEXT PRIMARY KEY,
  run_id TEXT NOT NULL,
  command TEXT NOT NULL,
  cwd TEXT NOT NULL,
  exit_code INTEGER,
  stdout_path TEXT,
  stderr_path TEXT,
  started_at TEXT NOT NULL,
  finished_at TEXT,
  timed_out INTEGER DEFAULT 0,
  FOREIGN KEY (run_id) REFERENCES runs(id)
);

```

14.6 traceability_items

```

CREATE TABLE traceability_items (
  id TEXT PRIMARY KEY,
  run_id TEXT NOT NULL,
  requirement_id TEXT NOT NULL,
  requirement_text TEXT NOT NULL,

```

```

acceptance_criteria_ids TEXT NOT NULL,
task_ids TEXT NOT NULL,
code_files TEXT NOT NULL,
test_cases TEXT NOT NULL,
test_results TEXT NOT NULL,
status TEXT NOT NULL,
FOREIGN KEY (run_id) REFERENCES runs(id)
);

```

14.7 settings

```

CREATE TABLE settings (
  key TEXT PRIMARY KEY,
  value TEXT NOT NULL,
  updated_at TEXT NOT NULL
);

```

15. Workflow Engine

15.1 Workflow Step Interface

```

interface WorkflowStep {
  name: string;
  execute(context: WorkflowContext): Promise<WorkflowContext>;
}

```

```

type WorkflowContext = {
  project: Project;
  run: Run;
  config: AiTeamConfig;
  paths: RunPaths;
  artifacts: Artifact[];
  state: Record<string, unknown>;
};

```

15.2 Workflow Types

Docs-only Workflow

```

create run
→ generate spec
→ generate plan
→ generate test matrix
→ generate final report

```

Assisted Workflow

```
create run
→ generate spec
→ generate plan
→ generate implementation prompt
→ generate final report
```

Semi-auto Workflow

```
create run
→ generate spec
→ generate plan
→ wait for approval
→ run selected AI CLI
→ collect diff
→ run tests
→ review
→ generate final report
```

Multi-agent Workflow

```
create run
→ BA Agent
→ PO Agent
→ PM Agent
→ Architect Agent
→ approval
→ Developer Agent
→ QA Agent
→ Reviewer Agent
→ Reporter Agent
```

15.3 Standard Workflow Implementation

```
const semiAutoWorkflow = [
  createRunStep,
  generateSpecStep,
  analyzeRepositoryStep,
  generatePlanStep,
  requireApprovalBeforeCodeStep,
  executeCodeAgentStep,
  collectDiffStep,
  runTestsStep,
  reviewStep,
```

```
generateTraceabilityStep,  
generateFinalReportStep  
];
```

16. AI CLI Adapter Layer

16.1 Adapter Interface

```
export interface AiCliAdapter {  
  name: AiAdapterName;  
  
  isAvailable(): Promise<boolean>;  
  
  runTask(input: AiTaskInput): Promise<AiTaskResult>;  
}
```

```
type AiAdapterName =  
  | "claude"  
  | "codex"  
  | "gemini"  
  | "aider";
```

```
type AiTaskInput = {  
  role: AgentRole;  
  prompt: string;  
  workingDir: string;  
  contextFiles?: string[];  
  outputLogPath: string;  
  timeoutSeconds: number;  
};
```

```
type AiTaskResult = {  
  success: boolean;  
  exitCode: number | null;  
  outputLogPath: string;  
  changedFiles: string[];  
  error?: string;  
};
```

16.2 Adapter Responsibilities

Each adapter should:

- check if CLI exists
- build command arguments

- run command in project directory
- stream output to log
- enforce timeout
- collect changed files
- return success/failure
- preserve raw logs

16.3 Adapter Strategy

The app should not hardcode deep behavior for every AI tool.

Use a simple adapter contract:

```
input prompt
+ working directory
+ timeout
+ context files
→ run external process
→ collect output
→ collect changed files
```

This keeps the product flexible if CLI tools change.

17. Shell Runner

17.1 Responsibilities

The shell runner is responsible for all external command execution.

It runs:

- AI CLI commands
- Git commands
- test commands
- build commands
- lint commands

17.2 ShellRunInput

```
type ShellRunInput = {
  command: string;
  args: string[];
  cwd: string;
  timeoutSeconds: number;
  stdoutPath: string;
  stderrPath: string;
```

```
env?: Record<string, string>;  
};
```

17.3 ShellRunResult

```
type ShellRunResult = {  
  exitCode: number | null;  
  stdoutPath: string;  
  stderrPath: string;  
  durationMs: number;  
  timedOut: boolean;  
};
```

17.4 Command Safety

Before running a command:

1. Check command against deny list.
2. Check current working directory is inside project root.
3. Apply timeout.
4. Log command.
5. Hide secrets from logs.
6. Confirm approval if command is risky.

Blocked examples:

```
sudo  
rm -rf /  
chmod 777  
curl | sh  
wget | sh  
mkfs  
dd if=
```

18. Git and Workspace Manager

18.1 Responsibilities

The Git manager handles:

- repository detection
- current branch detection
- uncommitted changes detection
- changed file collection
- diff generation
- patch creation
- rollback support

18.2 MVP Git Strategy

For MVP:

- do not auto-commit
- do not auto-push
- do not auto-merge
- only generate local diff
- user reviews manually

18.3 Later Git Strategy

Later versions can support:

- create feature branch
- create commit
- create pull request
- link PR to Jira
- read CI result

18.4 Git Snapshot

Before code generation:

```
save current git status
save current diff
mark run checkpoint
```

After code generation:

```
collect changed files
generate diff.patch
save changed-files.json
```

Rollback should require explicit user approval.

19. Repository Analyzer

19.1 Responsibilities

The repository analyzer detects:

- language
- framework
- build tool
- test framework
- source folders

- test folders
- config files
- migration tool
- API style
- existing project conventions

19.2 Java/Spring Boot Detection

Look for:

```
pom.xml
build.gradle
src/main/java
src/test/java
application.yml
application.properties
db/migration
```

Detect:

- Maven or Gradle
- Spring Boot version
- Java version
- JUnit
- Mockito
- Flyway
- Liquibase
- package structure
- controller/service/repository pattern

19.3 Node/NestJS Detection

Look for:

```
package.json
nest-cli.json
src/main.ts
src/**/*.module.ts
src/**/*.controller.ts
src/**/*.service.ts
```

Detect:

- npm/pnpm/yarn
- NestJS
- Jest
- Prisma
- TypeORM

19.4 React/Vite Detection

Look for:

```
package.json
vite.config.ts
src/main.tsx
src/App.tsx
```

Detect:

- React
- Vite
- Tailwind
- test framework
- component structure

19.5 Repository Analysis Output

```
{
  "language": "java",
  "framework": "spring-boot",
  "buildTool": "maven",
  "testFramework": "junit",
  "migrationTool": "flyway",
  "sourceDirs": ["src/main/java"],
  "testDirs": ["src/test/java"],
  "configFiles": ["src/main/resources/application.yml"],
  "detectedPatterns": [
    "controller-service-repository",
    "dto-based-api",
    "flyway-migrations"
  ]
}
```

20. Prompt Template System

20.1 Prompt Variables

Supported variables:

```
{{rawRequirement}}
{{clarifiedRequirement}}
{{acceptanceCriteria}}
{{businessRules}}
{{openQuestions}}
{{assumptions}}
```

```
{{projectSummary}}
{{repositoryAnalysis}}
{{technicalDesign}}
{{taskBreakdown}}
{{testMatrix}}
{{implementationPlan}}
{{changedFiles}}
{{diff}}
{{testResult}}
{{codingConvention}}
```

20.2 BA Agent Prompt Output

The BA Agent should output:

```
# Clarified Requirement

# Business Rules

# User Roles

# Main Flow

# Alternative Flows

# Edge Cases

# Acceptance Criteria

# Open Questions

# Assumptions

# Out of Scope
```

20.3 Architect Agent Prompt Output

The Architect Agent should output:

```
# Technical Design

## Summary

## Affected Modules

## API Design

## Database Design
```

```

## Service Flow

## Error Handling

## Security Considerations

## Performance Considerations

## Risks

## Implementation Plan

```

20.4 Developer Agent Prompt Rules

Developer Agent prompt should include:

- Follow existing project conventions.
- Make minimal changes.
- Do not edit unrelated files.
- Do not expose secrets.
- Do not modify production config unless explicitly requested.
- Add tests for new logic.
- Explain changed files.
- Stop if requirement is impossible or unclear.

20.5 QA Agent Prompt Output

```

# Test Matrix

| ID | Requirement | Scenario | Test Type | Expected Result | Priority |
|---|---|---|---|---|---|

```

20.6 Reviewer Agent Prompt Output

```

# Review Report

## Summary

## Requirement Coverage

## Code Quality

## Test Quality

## Security

```

```
## Risks

## Required Fixes

## Approval Status
```

21. Test Runner

21.1 Responsibilities

The test runner:

- runs configured test commands
- captures output
- detects pass/fail
- saves logs
- summarizes failures
- generates test result report

21.2 Test Command Config

```
{
  "commands": {
    "build": "mvn clean package -DskipTests",
    "unitTest": "mvn test",
    "integrationTest": "mvn verify",
    "lint": ""
  }
}
```

21.3 Test Result Output

```
# Test Result

## Summary

Status: FAILED

## Commands

### Build

Command: mvn clean package -DskipTests
Exit code: 0

### Unit Test
```

```
Command: mvn test
Exit code: 1
```

Failed Tests

- PasswordResetServiceTest.shouldRejectExpiredOtp

Failure Analysis

The expiration validation appears to use incorrect time comparison.

Suggested Fix

Review OTP expiration logic in PasswordResetService.

22. Review Engine

22.1 Inputs

The review engine reads:

- clarified requirement
- acceptance criteria
- technical design
- task breakdown
- diff patch
- changed files
- test result
- implementation notes

22.2 Outputs

The review engine generates:

- review report
- security review
- requirement coverage report

22.3 Review Checks

Checks:

- Does implementation match requirement?
- Are all acceptance criteria covered?
- Are tests added?
- Are there unrelated changes?
- Is error handling correct?
- Is authorization handled?

- Are secrets exposed?
- Is database migration safe?
- Is code maintainable?
- Are production config files modified?
- Are there risky commands or file changes?

23. Traceability Engine

23.1 Goal

The traceability engine maps:

```
requirement
→ acceptance criteria
→ task
→ code files
→ test cases
→ test result
```

23.2 Traceability JSON

```
{
  "items": [
    {
      "requirementId": "REQ-001",
      "requirementText": "User can request password reset using email OTP.",
      "acceptanceCriteriaIds": ["AC-001", "AC-002"],
      "taskIds": ["TASK-001", "TASK-002"],
      "codeFiles": [
        "src/main/java/com/example/auth/PasswordResetController.java",
        "src/main/java/com/example/auth/PasswordResetService.java"
      ],
      "testCases": [
        "PasswordResetServiceTest.shouldSendOtp",
        "PasswordResetServiceTest.shouldRejectExpiredOtp"
      ],
      "testResults": ["PASSED", "PASSED"],
      "status": "COVERED"
    }
  ]
}
```

23.3 Traceability Markdown

```
| Requirement | Acceptance Criteria | Task | Code Files | Tests | Status |
| --- | --- | --- | --- | --- | --- |
```

```
| REQ-001 | AC-001, AC-002 | TASK-001 | PasswordResetService.java |  
shouldSendOtp | Covered |
```

24. Integration Design

Integrations are optional.

The product should work without external integrations.

24.1 GitHub

MVP recommendation:

Use GitHub CLI first.

User runs:

```
gh auth login
```

Then the app can call:

```
gh pr create  
gh pr view  
gh run list
```

Alternative:

User enters GitHub token in local web UI.

Features:

- create PR
- read PR
- comment PR
- read CI result
- link PR to report

24.2 Jira

User enters in local web UI:

- Jira site URL
- email
- API token
- project key
- default issue type

Features:

- create epic
- create story
- create subtask
- comment final report
- update task status
- link PR

MVP can start with:

- generate Jira-ready markdown
- create tasks manually or via API later

24.3 Slack

Because the product is local-first, receiving Slack events is difficult without a public callback URL.

MVP Slack support should be:

- post progress update
- post final report
- post PR-ready notification

User enters:

- Slack bot token
- default channel ID

Receiving Slack commands can be added later using:

- ngrok
- Cloudflare Tunnel
- Tailscale Funnel
- future backend

24.4 Confluence / Notion

Later integrations:

- publish PRD
- publish technical design
- publish final report
- update docs

MVP can export Markdown only.

25. Security Design

25.1 Secret Protection

The system must prevent secrets from leaking into:

- prompts
- logs
- reports
- generated docs
- Git diffs

Rules:

- mask known secret values
- block reading `.env` unless user approves
- never include token values in prompt
- never print token values in logs
- store tokens in OS keychain

25.2 Protected Files

Default deny files:

```
.env
.env.*
*.pem
*.key
credentials.json
application-prod.yml
application-production.yml
```

Default warn files:

```
pom.xml
build.gradle
package.json
Dockerfile
.github/workflows/*
```

25.3 Dangerous Commands

Block commands containing:

```
sudo
rm -rf /
chmod 777
```

```
curl | sh
wget | sh
mkfs
dd if=
```

25.4 Approval Gates

Default approval gates:

- before code generation
- before applying patch
- before rollback
- before commit
- before PR creation
- before Jira update
- before Slack notification

26. Error Handling

26.1 Error Categories

```
CONFIG_ERROR
PROJECT_NOT_INITIALIZED
AI_CLI_NOT_FOUND
AI_CLI_FAILED
GIT_ERROR
TEST_FAILED
COMMAND_TIMEOUT
DANGEROUS_COMMAND_BLOCKED
PROTECTED_FILE_MODIFIED
CREDENTIAL_MISSING
INTEGRATION_FAILED
WORKFLOW_FAILED
```

26.2 Error Output Example

```
# Workflow Failed

Reason: AI_CLI_NOT_FOUND

The selected agent "codex" is not available on this machine.

Suggested actions:

1. Install Codex CLI.
```

2. Change default developer agent in Settings.
3. Run: aiteam doctor

27. Logging

27.1 Log Types

The system should create:

- workflow log
- shell command log
- AI agent output log
- test log
- integration log
- error log

27.2 Log Location

```
.ai-team/runs/<runId>/logs/  
  workflow.log  
  shell.log  
  agent.log  
  test.log  
  integration.log  
  error.log
```

27.3 Log Redaction

Before writing logs:

- remove API tokens
- mask environment variable values
- mask Authorization headers
- mask secret-like strings
- avoid storing full `.env` content

28. MVP Implementation Plan

Phase 1: CLI + Local Data Foundation

Build:

- monorepo setup
- CLI app
- `init`
- `doctor`
- config loader

- `.ai-team` folder creation
- SQLite setup
- run folder creation

Phase 2: Local Web UI Foundation

Build:

- `aiteam ui`
- Fastify local server
- React local web UI
- settings page
- run history page
- run detail page
- docs viewer

Phase 3: Requirement Docs Workflow

Build:

- `run --mode docs-only`
- BA Agent prompt
- Architect Agent prompt
- PM Agent prompt
- QA Agent prompt
- Markdown artifact generator
- artifact metadata storage

Outputs:

- clarified requirement
- acceptance criteria
- technical design
- task breakdown
- test matrix
- final report

Phase 4: Repository Analyzer

Build:

- Java/Spring Boot detector
- Node/NestJS detector
- React/Vite detector
- project summary generator
- existing pattern summary

Phase 5: AI CLI Adapter

Build:

- shell runner
- adapter interface
- one adapter first, preferably Codex or Claude
- changed files collection
- diff patch generation
- log streaming

Phase 6: Test and Review

Build:

- test runner
- test result report
- review prompt
- security review prompt
- traceability generator

Phase 7: Integrations

Build later:

- GitHub via `gh` CLI
- Jira token-based integration
- Slack post-message integration
- Confluence/Notion export

29. Recommended MVP Scope

MVP should include:

CLI:

- `init`
- `ui`
- `doctor`
- `run`
- `spec`
- `plan`
- `report`

Local Web:

- `settings`
- `prompt templates`
- `run history`
- `docs viewer`
- `logs viewer`

Core:

- requirement clarification
- acceptance criteria
- technical design
- task breakdown
- test matrix
- final report

MVP+ should include:

- one AI CLI adapter
- code generation
- diff collection
- test runner
- review report
- traceability matrix

Avoid in MVP:

- cloud backend
- desktop app
- login
- billing
- full Jira automation
- full Slack bot
- automatic deployment

30. Example End-to-End Flow

User starts local UI:

```
aiteam ui
```

User enters:

```
Thêm API export invoice CSV theo hotelId, date range, status.
```

System generates:

1. clarified requirement
2. acceptance criteria
3. technical design
4. API design
5. task breakdown

6. test matrix
7. implementation prompt

If user approves code generation:

8. selected AI CLI runs
9. changed files collected
10. diff patch generated
11. tests run
12. review report generated
13. final report generated
14. traceability matrix generated

Final output:

```
.ai-team/runs/2026-06-22-export-invoice-csv/  
  requirement/  
  design/  
  tasks/  
  tests/  
  implementation/  
  review/  
  report/  
  logs/
```

31. Final Technical Statement

Local AI Software Team should be built as:

```
CLI-first  
Local-web-assisted  
Local-first  
No desktop app  
No backend cloud  
No login required  
BYO AI CLI  
BYO integration tokens  
File-based artifacts  
SQLite metadata  
Git-safe workflow  
Human approval gates  
Prompt-template-driven agent roles
```

The first strong version should deliver:


```
rough requirement
→ clarified requirement
→ acceptance criteria
→ design
→ task breakdown
→ test matrix
→ implementation prompt
→ final report
```

The second strong version should add:

```
AI CLI execution
→ code patch
→ test result
→ review report
→ traceability matrix
```

The long-term version should become:

```
a local AI software team that coordinates existing AI coding tools and
optional external integrations to move software work from vague requirement
to reviewed delivery package.
```